# Chapter 7: Reliable Data Delivery

Reliability guarantees that Kafka gives.

1) Kafka provides ordering guarantees - if message A was sent before message B then message A will have a smaller offset than message B → (provided max-inflight-requests = 1)

2) Produced messages are acks=all, written to all in-sync replicas

3)

(Replication)

A replica is said to be in-sync with leader:

→ Has active session with zookeeper and has sent heartbeat in last 6 seconds

→ Has fetched messages from last 10 seconds from the leader.

→ Fetched most recent message from the leader within 10 seconds of it being written to Kafka

A lot of times an in-sync replica falls out into out of sync because of either GC or large request size leading to flipping.

Well if in-sync replica is slow, then producer writes can take a hit, but once it goes out of sync, no longer contributes to performance.

{acks=all} [safest option].

## Unclean Leader Election

Imagine, a broker has 5 replicas,

3 are in-sync and 2 out-of-sync

Now imagine all 3 in-sync replica go down,

What happens?

Leader Election triggers →

A config → unclean.leader-election-enable.

If this is set to true it can elect any out of sync replica to be leader, which is not safe

So by default it is set to false. If all in-sync replicas are down, then to maintain order guarantee we make that partition unavailable

↳ we throw an Error (NotEnoughReplicasException)

[Few trade offs we do]

↳ zookeeper-session-timeout-ms → max time a Replica can go without sending a zookeeper heartbeat before considering its dead.

replica.~~max~~lag-time.max.ms → max time replica can go without fetching for messages before going out of sync

Few Configs and metrics,
which you can check if you want to ensure Reliability

auto.offset.reset → Earliest, Latest, when a new consumer joins & the grp coordinator does not have its partition mapping

enable.auto-commit → We saw setting this to true is risky as we may miss out on messages which got committed but not processed.

(chapter 4)

→ if we set this to false and do our commit manually much safer and reliable

Kafka gives us JMX metrics → using which we can monitor in production.

For Producers we get → (ERROR_RATE, RETRY_RATE) graphs.

For Consumers → we can check Lag,

For Brokers → we get metric of

(Failed Produce Reqs/sec → on producer
Failed Fetch Reqs/sec → on consumer)